

MediBook Phase 2

Doctor Module with Approval Workflow

COMPLETE LEARNING GUIDE

State Machine · Pagination · Dynamic Queries
Role-Based Controllers · Audit Trail · Real Testing

What You Built in Phase 2

DoctorStatus enum (4-state workflow)

Doctor entity with One-to-One User link

DoctorRepository with derived queries + Specifications

7 DTOs (request/response variants)

DoctorMapper for entity-DTO conversions

DoctorService with state machine validation

3 controllers (Doctor, Admin, Public)

Updated SecurityConfig with method-based access

20 endpoints tested end-to-end in Postman

Audit trail (approved_by, rejected_by, timestamps)

Pagination, sorting, filtering with Spring Data

Dynamic search by name and specialization

Stack: Java 21 · Spring Boot 3.5 · MySQL 8 · JWT · Spring Security

Repository: github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook

Table of Contents

PART I DESIGN & ARCHITECTURE

1. Business Requirements
2. Doctor Onboarding Workflow
3. State Machine Design
4. Module LLD
5. API Design (All Endpoints)

PART II DATA LAYER

6. DoctorStatus Enum
7. Doctor Entity Deep Dive
8. Entity Relationships (@OneToOne)
9. Database Indexes Strategy
10. DoctorRepository
11. JPA Specifications Explained

PART III DTO & MAPPING LAYER

12. Why 7 Different DTOs?
13. Request DTOs (Create, Update, Approve, Reject)
14. Response DTOs (Full, Public, Summary)
15. DoctorMapper Pattern
16. JPA Dirty Checking

PART IV SERVICE LAYER

17. Service Interface Design
18. State Machine Implementation
19. Audit Trail Logic
20. Resource Hiding (404 over 403)
21. Transaction Management

PART V CONTROLLER LAYER

22. Three Controllers Pattern
23. DoctorController (Self-Service)
24. AdminDoctorController
25. PublicDoctorController
26. Pagination with Pageable

PART VI SECURITY UPDATES

27. SecurityConfig Changes
28. Method-Level vs Class-Level @PreAuthorize
29. Regex in URL Patterns

PART VII TESTING & VERIFICATION

30. Test Strategy
31. Complete Postman Test Suite
32. MySQL Verification

PART VIII REFERENCE

33. All Endpoints Cheat Sheet
34. Common Mistakes & Fixes
35. Interview Questions for Phase 2

36. What's Next: Phase 3 Preview

PART I · DESIGN & ARCHITECTURE

1. Business Requirements

Phase 2 implements the doctor onboarding journey. Doctors self-register, create their professional profile, and an admin reviews and approves before they become visible to patients.

Functional Requirements

- Doctor self-registration with role=DOCTOR
- Doctor creates professional profile (specialization, license, fee, etc.)
- Profile starts in PENDING_APPROVAL state
- Admin reviews pending doctors
- Admin approves OR rejects (with reason)
- Rejected doctors can resubmit IF admin allows it
- Only APPROVED doctors visible in public listings
- Admin can suspend/reactivate approved doctors
- Public browsing without authentication (browse before signup)
- Search and filter by name, specialization

Non-Functional Requirements

- **Audit trail:** Track who approved/rejected and when
- **Performance:** Pagination on all list endpoints
- **Security:** Role-based access (DOCTOR vs ADMIN vs PUBLIC)
- **Privacy:** Hide pending/rejected/suspended doctors from public
- **Validation:** Bean Validation on all input

INTERVIEW POINT » Interview Q: "Why have an approval workflow instead of immediate doctor visibility?"

A: "Healthcare is a regulated domain. Verifying credentials (medical license, qualifications) before allowing public-facing presence is a legal and ethical requirement. Patients trust the platform to vet doctors. The approval workflow adds a verification step that mirrors real-world hospital onboarding."

2. Doctor Onboarding Workflow

End-to-End Workflow

STEP 1: Doctor self-registers as USER
POST /auth/register
body: { email, password, fullName, role: DOCTOR }
-> User account created

STEP 2: Doctor logs in -> gets JWT
POST /auth/login

STEP 3: Doctor creates professional profile
POST /doctors/me
body: { specialization, qualification, license, fee, ... }
-> Doctor record created with status=PENDING_APPROVAL

STEP 4: Admin reviews pending doctors
GET /admin/doctors?status=PENDING_APPROVAL

STEP 5: Admin decides:

- a) APPROVE: PUT /admin/doctors/{id}/approve
-> status=APPROVED
-> doctor publicly visible
- b) REJECT: PUT /admin/doctors/{id}/reject
body: { rejectionReason, resubmissionAllowed }
-> status=REJECTED

STEP 6: If REJECTED with resubmission allowed:
Doctor sees: GET /doctors/me
Doctor updates: PUT /doctors/me/resubmit
-> status back to PENDING_APPROVAL

STEP 7: Public can browse APPROVED doctors:

- GET /doctors (anyone)
- GET /doctors/{id} (anyone)
- GET /doctors/search?name=Smith (anyone)

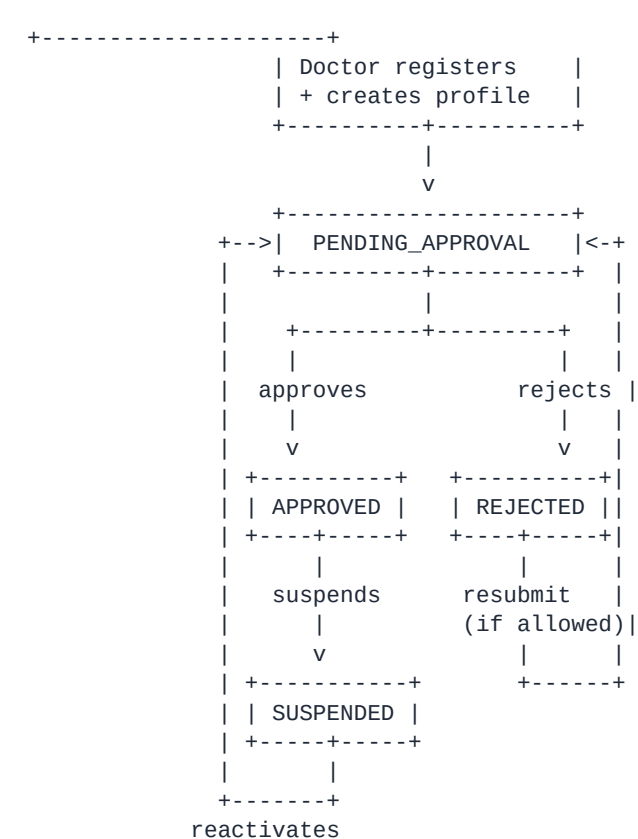
STEP 8: Admin can later:

- PUT /admin/doctors/{id}/suspend (status=SUSPENDED)
- PUT /admin/doctors/{id}/reactivate (back to APPROVED)

3. State Machine Design

A state machine defines valid transitions between states. Our doctor module has 4 states and carefully controlled transitions. Each transition is validated in the service layer.

State Machine Diagram



Valid Transitions

From	To	Trigger	Validation
(none)	PENDING_APPROVAL	Doctor creates profile	Verify DOCTOR role, unique license
PENDING_APPROVAL	APPROVED	Admin approves	Admin role required
PENDING_APPROVAL	REJECTED	Admin rejects	Reason required, optional resubmit
REJECTED	PENDING_APPROVAL	Doctor resubmits	resubmissionAllowed must be true
APPROVED	SUSPENDED	Admin suspends	Admin role required
SUSPENDED	APPROVED	Admin reactivates	Admin role required

Invalid Transitions (Blocked by Service)

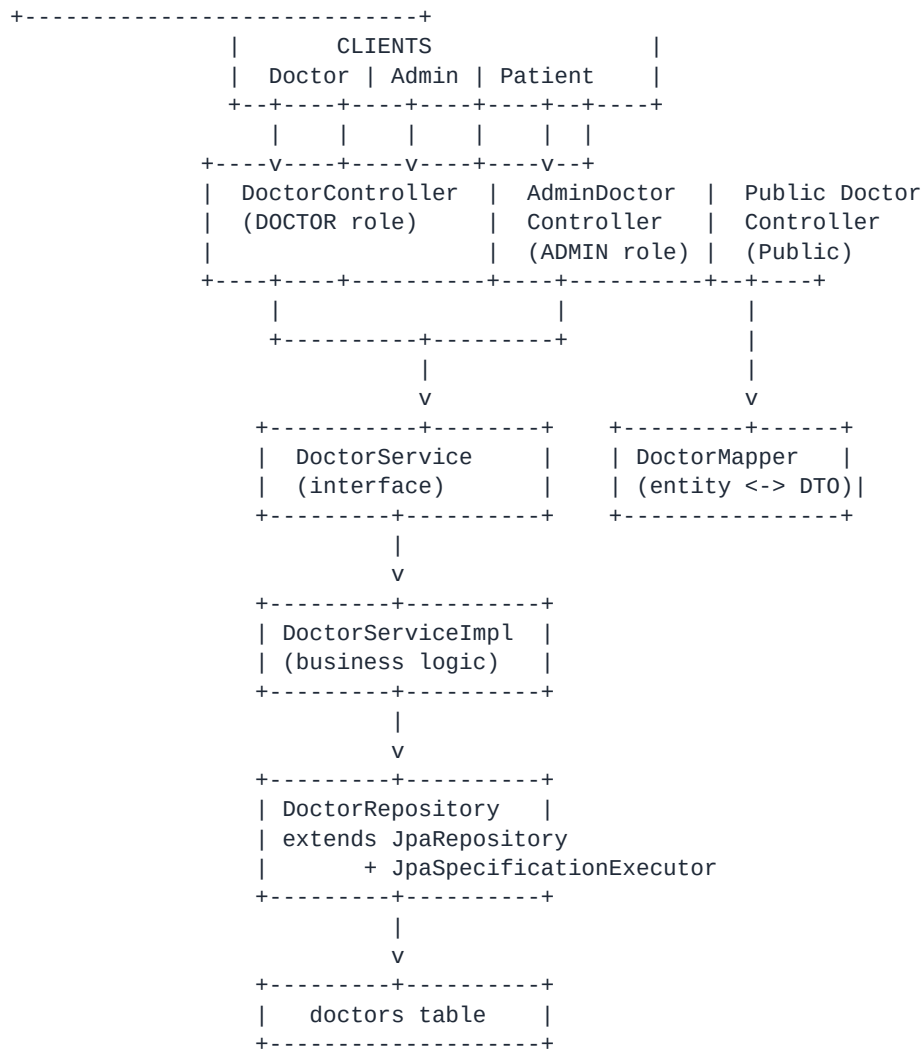
- APPROVED -> REJECTED (must suspend first)
- REJECTED -> APPROVED (must resubmit first)
- APPROVED -> PENDING_APPROVAL (no use case)
- SUSPENDED -> PENDING_APPROVAL (no use case)

INTERVIEW POINT » Interview Q: "How do you enforce state machine rules?"

A: "In the service layer, before any state change, I validate the current state. For example, `approveDoctor()` checks `if status == PENDING_APPROVAL`. If not, throws `BadRequestException`. This centralized validation in the service prevents invalid transitions regardless of which API endpoint or future caller is used. Some teams extract this into a separate `StateMachine` class."

4. Module LLD

Phase 2 LLD



5. API Design (All Endpoints)

Doctor Endpoints (DOCTOR role required)

Method	Endpoint	Purpose
POST	/doctors/me	Create professional profile
GET	/doctors/me	View own profile
PUT	/doctors/me	Update own profile
PUT	/doctors/me/resubmit	Resubmit after rejection

Admin Endpoints (ADMIN role required)

Method	Endpoint	Purpose
GET	/admin/doctors	List doctors by status
GET	/admin/doctors/{id}	View any doctor details
PUT	/admin/doctors/{id}/approve	Approve doctor
PUT	/admin/doctors/{id}/reject	Reject with reason
PUT	/admin/doctors/{id}/suspend	Suspend approved doctor
PUT	/admin/doctors/{id}/reactivate	Reactivate suspended

Public Endpoints (No auth required)

Method	Endpoint	Purpose
GET	/doctors	Browse approved doctors
GET	/doctors/{id}	View doctor details
GET	/doctors/search	Search by name/specialization
GET	/doctors/specializations	Get filter options

TIP » Total: 14 endpoints in Phase 2, all secured appropriately by role.

PART II · DATA LAYER

6. DoctorStatus Enum

Why this matters » Enums provide compile-time safety. With a String column, typos like "PENDIG" would compile but fail at runtime. With enum, the compiler catches errors. Plus we get IDE autocomplete and safe refactoring.

src/main/java/com/medibook/medibookservice/enums/DoctorStatus.java

```
package com.medibook.medibookservice.enums;

public enum DoctorStatus {
    PENDING_APPROVAL,
    APPROVED,
    REJECTED,
    SUSPENDED
}
```

Storage in Database

Using `@Enumerated(EnumType.STRING)` on the entity field, JPA stores the enum NAME as text:

Database storage

```
status VARCHAR(30) NOT NULL
-- Values: 'PENDING_APPROVAL', 'APPROVED', 'REJECTED', 'SUSPENDED'
```

COMMON MISTAKE » NEVER use EnumType.ORDINAL - it stores 0, 1, 2, 3. If you reorder or insert a new enum value, ALL existing data becomes corrupted because the indices shift. Always STRING.

7. Doctor Entity Deep Dive

Table Schema

doctors table

DOCTORS TABLE				
id	BIGINT	PK AUTO		
user_id	BIGINT	FK UNIQUE NOT NULL		
--- PROFESSIONAL INFO ---				
full_name	VARCHAR(100)	NOT NULL		
phone_number	VARCHAR(15)			
specialization	VARCHAR(50)	NOT NULL	[INDEX]	
qualification	VARCHAR(200)	NOT NULL		
experience_years	INT	NOT NULL		
license_number	VARCHAR(50)	UNIQUE NOT NULL	[INDEX]	
--- PROFILE INFO ---				
bio	TEXT			
profile_picture_url	VARCHAR(500)			
--- BUSINESS INFO ---				
consultation_fee	DECIMAL(10,2)			
languages_spoken	VARCHAR(200)			
--- APPROVAL WORKFLOW ---				
status	VARCHAR(30)	NOT NULL	[INDEX]	
rejection_reason	TEXT			
resubmission_allowed	BOOLEAN	DEFAULT FALSE		
--- AUDIT TRAIL ---				
approved_by_user_id	BIGINT	NULLABLE		
approved_at	TIMESTAMP	NULLABLE		
rejected_by_user_id	BIGINT	NULLABLE		
rejected_at	TIMESTAMP	NULLABLE		
--- TIMESTAMPS ---				
created_at	TIMESTAMP	AUTO		
updated_at	TIMESTAMP	AUTO		

Total: 19 columns + 4 indexes

Critical Annotations Explained

@OneToOne(fetch = FetchType.LAZY, optional = false)

User relationship

```
@OneToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "user_id", referencedColumnName = "id",
            nullable = false, unique = true)
private User user;
```

Setting	Effect
fetch = LAZY	Don't auto-load User; load only when getUser() called
optional = false	User is REQUIRED (mirrors DB NOT NULL)
name = user_id	Column name in doctors table
referencedColumnName = id	Points to users.id (FK)
nullable = false	Enforces NOT NULL at JPA level
unique = true	Enforces One-to-One (each user has at most 1 doctor)

BigDecimal for Money

Fee field

```
@Column(name = "consultation_fee", precision = 10, scale = 2)
private BigDecimal consultationFee;
```

COMMON MISTAKE » NEVER use double or float for money!

double: $0.1 + 0.2 = 0.30000000000000004$ (rounding error)

BigDecimal: EXACT precision, no rounding errors. Industry standard for financial calculations.

Default Status via Java

Default status

```
@Enumerated(EnumType.STRING)
@Column(name = "status", nullable = false, length = 30)
private DoctorStatus status = DoctorStatus.PENDING_APPROVAL;
//                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//                               Java-level default
```

Why this matters » Setting default in Java code (not DB DEFAULT clause) makes the default visible to anyone reading the entity. DB defaults can be silently overridden by migration tools and aren't always obvious.

8. Entity Relationships (@OneToOne)

There are 4 main JPA relationship types. Phase 2 uses One-to-One.

JPA Relationships

1. @OneToOne - User ----- Doctor profile
(One user has at most one doctor profile)
2. @OneToMany - Doctor -----< Appointments
(One doctor has many appointments)
3. @ManyToOne - Appointments >----- Doctor
(Many appointments belong to one doctor)
(Inverse of OneToMany)
4. @ManyToMany - Doctors >-----< Specializations
(Doctors can have multiple specialties,
each specialty has multiple doctors)
(Uses a JOIN TABLE)

Unidirectional vs Bidirectional

Direction options

UNIDIRECTIONAL (our choice):

Doctor knows about User, but User doesn't know about Doctor

Doctor.java:

```
@OneToOne
private User user;
```

User.java:

(no reference to Doctor)

BIDIRECTIONAL (more complex):

Both sides know each other

Doctor.java:

```
@OneToOne
private User user;
```

User.java:

```
@OneToOne(mappedBy = "user")
private Doctor doctor;
```

Why this matters » Unidirectional is simpler. We chose it because we always start from Doctor (logged-in doctor views profile, admin browses doctors). We rarely go from User -> Doctor. If we did, we'd add the inverse side later.

9. Database Indexes Strategy

Why this matters » Indexes dramatically speed up SELECT queries on indexed columns. Without indexes, MySQL does a full table scan. With indexes, it uses a B-tree lookup. Trade-off: indexes slow down INSERT/UPDATE slightly because the index must also be maintained.

Our Index Choices

Index	Column(s)	Why
idx_doctor_license	license_number UNIQUE	Prevent duplicates + fast lookup
idx_doctor_specialization	specialization	Patients filter by specialty
idx_doctor_status	status	Admin filters by status (very common!)
idx_doctor_user	user_id UNIQUE	FK lookup + enforce One-to-One

Index Code in Entity

Index definitions

```
@Table(name = "doctors", indexes = {
    @Index(name = "idx_doctor_license",
        columnList = "license_number", unique = true),
    @Index(name = "idx_doctor_specialization",
        columnList = "specialization"),
    @Index(name = "idx_doctor_status",
        columnList = "status"),
    @Index(name = "idx_doctor_user",
        columnList = "user_id", unique = true)
})
```

Performance Impact

Index magic

Without index on 'status':
SELECT * FROM doctors WHERE status = 'APPROVED'
-> Full table scan, $O(n)$
-> 1M rows = ~1M comparisons

With index on 'status':
SELECT * FROM doctors WHERE status = 'APPROVED'
-> B-tree lookup, $O(\log n)$
-> 1M rows = ~20 comparisons

That is 50,000x faster!

TIP » Indexes help WHERE, ORDER BY, JOIN, and GROUP BY. They DON'T help SELECT *. Index columns you FILTER by, SORT by, or JOIN on - not columns you just display.

10. DoctorRepository

DoctorRepository.java

```
@Repository
public interface DoctorRepository extends JpaRepository<Doctor, Long>,
                                           JpaSpecificationExecutor<Doctor> {

    // Find by user
    Optional<Doctor> findByUser(User user);
    Optional<Doctor> findById(Long userId);

    // Existence checks
    boolean existsByUser(User user);
    boolean existsByLicenseNumber(String licenseNumber);

    // Status queries
    Page<Doctor> findByStatus(DoctorStatus status, Pageable pageable);
    long countByStatus(DoctorStatus status);

    // Combined queries
    Page<Doctor> findByStatusAndSpecialization(
        DoctorStatus status, String specialization, Pageable pageable);

    // Custom @Query
    @Query("SELECT d FROM Doctor d " +
        "WHERE d.status = :status " +
        "AND LOWER(d.fullName) LIKE LOWER(CONCAT('%', :name, '%'))")
    Page<Doctor> searchApprovedDoctorsByName(
        @Param("status") DoctorStatus status,
        @Param("name") String name,
        Pageable pageable);

    // Distinct values
    @Query("SELECT DISTINCT d.specialization FROM Doctor d " +
        "WHERE d.status = :status " +
        "ORDER BY d.specialization ASC")
    List<String> findDistinctSpecializationsByStatus(
        @Param("status") DoctorStatus status);
}
```

Three Query Styles Used

Style	Example	When to Use
Derived (method name)	findById(Long id)	Simple queries
@Query (JPQL)	Custom JPQL string	Complex queries
Specifications	Built at runtime	Dynamic filters

11. JPA Specifications Explained

Why this matters » Sometimes filters are optional and combine dynamically. "Search doctors WHERE name LIKE X AND (optionally) specialization = Y AND (optionally) fee BETWEEN A AND B". Writing every combination as a method explodes. Specifications build the query at runtime based on what filters are present.

How Specifications Work

Specification pattern

Specification = a predicate (a condition for WHERE clause)

You build them:

```
Specification<Doctor> byStatus = (root, query, cb) ->
    cb.equal(root.get("status"), DoctorStatus.APPROVED);
```

```
Specification<Doctor> bySpec = (root, query, cb) ->
    cb.equal(root.get("specialization"), "Cardiology");
```

You combine them:

```
Specification<Doctor> combined = byStatus.and(bySpec);
```

You execute:

```
Page<Doctor> results = doctorRepository.findAll(combined, pageable);
```

JPA generates SQL:

```
SELECT * FROM doctors
WHERE status = 'APPROVED' AND specialization = 'Cardiology'
```

Beauty: each spec is independent. Mix and match at runtime!

Phase 2 Doesn't Use Specs Yet - Future Phases Will

We extended JpaSpecificationExecutor in our repository but didn't build Specifications yet. We will in Phase 3+ when we need advanced filtering like "doctors with experience > 5 years AND fee between 1000-2000 AND speaks Hindi or Tamil". The extension is in place: we'll use it later.

INTERVIEW POINT » Interview Q: "When would you use Specifications vs derived queries vs @Query?"

A: "Derived queries for simple, fixed conditions. @Query for complex but static SQL. Specifications for dynamic, optional filters that combine at runtime. For our doctor search, if I have 5 optional filters, derived queries would explode into $2^5 = 32$ methods. Specifications handle this elegantly with one method."

PART III · DTO & MAPPING LAYER

12. Why 7 Different DTOs?

Phase 2 has 7 DTOs. Each serves a specific purpose - it's not over-engineering.

DTO	Used In	Why Separate
DoctorProfileCreateRequest	POST /doctors/me	Includes licenseNumber (set once)
DoctorProfileUpdateRequest	PUT /doctors/me	Excludes licenseNumber (immutable)
DoctorApprovalRequest	PUT /admin/.../approve	Just notes field
DoctorRejectionRequest	PUT /admin/.../reject	Requires reason + resubmit flag
DoctorResponse	Authenticated GETs	Full details including audit
DoctorPublicResponse	Public listings	Hides phone, license, audit
DoctorSummaryResponse	List views	Minimal data for performance

The Key Principle

Why this matters » Different consumers need different views. Patients shouldn't see a doctor's phone or license number. Admins need everything. List views need minimal data for performance. ONE big DTO would force all consumers to ignore data they shouldn't see - a security and performance anti-pattern.

13. Request DTOs

DoctorProfileCreateRequest (with full validation)

Create request with `@Valid` annotations

```
public class DoctorProfileCreateRequest {

    @NotBlank(message = "Full name is required")
    @Size(min = 2, max = 100)
    private String fullName;

    @NotBlank @Size(max = 50)
    private String specialization;

    @NotBlank @Size(max = 200)
    private String qualification;

    @NotNull @Min(0) @Max(70)
    private Integer experienceYears;

    @NotBlank @Size(max = 50)
    private String licenseNumber;           // <- Required on create

    @DecimalMin("0.0") @DecimalMax("99999999.99")
    private BigDecimal consultationFee;

    // ... other fields
}
```

DoctorProfileUpdateRequest (no licenseNumber)

License number is intentionally **excluded** from updates. Once a doctor is approved, the license is locked - they can't change it via the API. This enforces a business rule.

DoctorRejectionRequest (reason mandatory)

Rejection request

```
public class DoctorRejectionRequest {

    @NotBlank(message = "Rejection reason is required")
    @Size(min = 10, max = 1000)
    private String rejectionReason;

    @NotNull
    private Boolean resubmissionAllowed;
}
```

TIP » Always make rejection reasons MANDATORY. It helps the doctor understand what to fix. And the 10-char minimum prevents lazy "no" rejections.

14. Response DTOs

The 3-Tier Response Pattern

Response variants

PUBLIC (least info):

```
+-----+
| DoctorPublicResponse |
+-----+
| id                   |
| fullName             |
| specialization       |
| qualification        |
| experienceYears      |
| bio                  |
| profilePictureUrl    |
| consultationFee       |
| languagesSpoken      |
+-----+
```

AUTHENTICATED (more):

```
+-----+
| DoctorResponse      |
+-----+
| id                   |
| userId, email        |
| fullName             |
| phoneNumber          | <- Hidden from public
| specialization       |
| qualification        |
| experienceYears      |
| licenseNumber        | <- Hidden from public
| bio, profilePic      |
| consultationFee       |
| languagesSpoken      |
| status               | <- Workflow info
| rejectionReason       |
| resubmissionAllowed  |
| approvedByUserId     | <- Audit
| approvedAt           |
| rejectedByUserId     |
| rejectedAt           |
| createdAt, updatedAt |
+-----+
```

@JsonInclude(NON_NULL)

Skip null fields in JSON

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public class DoctorResponse {
    // ... fields
}
```

Without this annotation, a JSON response for a PENDING_APPROVAL doctor would include **rejectionReason: null, approvedAt: null, approvedByUserId: null** - cluttered. With `@JsonInclude(NON_NULL)`, null fields are simply omitted from the JSON. Cleaner output.

15. DoctorMapper Pattern

Why this matters » Mapping between Entity and DTO is repetitive. Doing it inline in services creates 50+ lines of field assignments in every method. The Mapper pattern centralizes this in one place - DRY principle.

Three Conversion Methods

Method	Direction	Use Case
toEntity(req, user)	DTO -> Entity	Doctor creates profile
updateEntity(doc, req)	DTO -> Entity (in-place)	Doctor updates profile
toResponse(doctor)	Entity -> DTO (full)	Authenticated response
toPublicResponse(doctor)	Entity -> DTO (public)	Patient browsing
toSummaryResponse(doctor)	Entity -> DTO (minimal)	List views
toPublicSummaryResponse(doctor)	Entity -> DTO (minimal+public)	Public list views

In-Place Updates (Critical Pattern)

In-place update

```
public void updateEntity(Doctor doctor, DoctorProfileUpdateRequest request) {
    if (doctor == null || request == null) return;

    doctor.setFullName(request.getFullName());
    doctor.setPhoneNumber(request.getPhoneNumber());
    doctor.setSpecialization(request.getSpecialization());
    // ... etc.

    // licenseNumber is INTENTIONALLY NOT updated
}
```

WARNING » Method returns VOID and modifies the existing entity. Why? Because the existing entity is **managed by JPA**. JPA's dirty checking will auto-save these changes at transaction commit. If we returned a NEW entity, we'd lose this and potentially create duplicates.

16. JPA Dirty Checking

One of the most powerful JPA features. And it confuses beginners.

What is Dirty Checking?

Dirty checking explained

WITHIN a @Transactional method:

1. Load entity from DB
`Doctor doctor = doctorRepo.findById(1L).get();`
2. JPA marks doctor as "MANAGED"
(It tracks the original state in memory)
3. Modify the entity
`doctor.setStatus(DoctorStatus.APPROVED);`
4. NO `doctorRepo.save()` needed!
JPA tracked the change.
5. Transaction commits
JPA compares current state to original
Detects status changed
Auto-generates: `UPDATE doctors SET status='APPROVED' WHERE id=1`

That is dirty checking!

Why It Matters

- Less code: no explicit `save()` calls
- Efficient: only changed fields trigger UPDATE
- Atomic: all changes in one transaction commit together
- Safer: prevents accidental duplicates

COMMON MISTAKE » Dirty checking ONLY works:

1. Inside a @Transactional method
2. On an entity loaded by JPA (not `new()`)

If you load an entity outside @Transactional and try to modify it, changes are LOST.

INTERVIEW POINT » Interview Q: "Why don't you call `repository.save()` after modifying an entity?"

A: "JPA dirty checking. Within a @Transactional method, JPA tracks the original state of managed entities. Any modification is automatically persisted at transaction commit. Calling `save()` explicitly is redundant and can cause issues with managed entities. Save is only needed for NEW entities being persisted for the first time."

PART IV · SERVICE LAYER

17. Service Interface Design

Why this matters » Separating the interface from implementation enables Dependency Inversion (SOLID's D). Controllers depend on the interface (DoctorService), not the implementation (DoctorServiceImpl). Spring injects the impl at runtime. This makes testing easier (mock the interface) and lets us swap implementations without changing callers.

Method Organization

Service method groups

DoctorService interface

```

      |
+-----+-----+-----+
|         |         |         |
DOCTOR    ADMIN    PUBLIC
self-service operations queries
|         |         |
createProfile() approve() getApprovedDoctors()
updateMyProfile() reject() getApprovedDoctorById()
getMyProfile() suspend() searchApprovedDoctors()
resubmitProfile() reactivate() getDistinctSpecializations()
                        getIdForAdmin()
                        getIdByStatus()
```

TIP » Organizing by AUDIENCE (doctor/admin/public) instead of by HTTP verb makes the service easier to understand. Anyone reading the code immediately sees who can call what.

18. State Machine Implementation

Pattern: Validate Then Modify

State transition pattern

```
@Override
@Transactional
public DoctorResponse approveDoctor(Long doctorId,
                                    DoctorApprovalRequest request,
                                    User adminUser) {
    Doctor doctor = findDoctorById(doctorId);

    // STEP 1: Validate current state
    if (doctor.getStatus() != DoctorStatus.PENDING_APPROVAL) {
        throw new BadRequestException(
            "Doctor can only be approved from PENDING_APPROVAL status. " +
            "Current: " + doctor.getStatus());
    }

    // STEP 2: Apply state change
    doctor.setStatus(DoctorStatus.APPROVED);
    doctor.setApprovedByUserId(adminUser.getId());
    doctor.setApprovedAt(LocalDateTime.now());

    // STEP 3: Clear stale data from previous state
    doctor.setRejectionReason(null);
    doctor.setResubmissionAllowed(false);
    doctor.setRejectedByUserId(null);
    doctor.setRejectedAt(null);

    // (No save() needed - dirty checking handles it)
    return doctorMapper.toResponse(doctor);
}
```

Why Clear Stale Data?

When a doctor is approved after a previous rejection, we need to clear the old rejection info. Otherwise an APPROVED doctor would still show "rejectionReason: ..." from the past. Clean state transitions prevent stale data confusion.

19. Audit Trail Logic

Why this matters » Healthcare apps need accountability. Who approved this doctor? When? Who rejected? For what reason? Audit trails are critical for compliance (HIPAA, GDPR, etc.), dispute resolution, and security investigations.

Our Audit Fields

Field	Captured When	Purpose
approved_by_user_id	Admin approves	WHO approved
approved_at	Admin approves	WHEN approved
rejected_by_user_id	Admin rejects	WHO rejected
rejected_at	Admin rejects	WHEN rejected
created_at	Auto on insert	Profile creation time
updated_at	Auto on any update	Last modified time

TIP » Why store user_id instead of full user object? Storing just the ID avoids data duplication and stays valid even if the admin's email/name changes later. To get details, JOIN on users.id when needed.

Going Further: Spring Data Auditing

For automatic createdBy/updatedBy fields, you can use @CreatedBy and @LastModifiedBy with an AuditorAware bean that returns the current authenticated user. We chose manual tracking for explicit control - approved_by is different from updated_by, since updates by the doctor shouldn't be counted as approvals.

20. Resource Hiding (404 over 403)

The Security Question

The choice

Patient requests: GET /doctors/123

Case 1: Doctor 123 doesn't exist
Response: 404 Not Found

Case 2: Doctor 123 exists but is REJECTED
Two choices:
a) 403 Forbidden "Doctor not visible"
b) 404 Not Found (same as case 1)

Which is better?

Why We Chose 404

Resource hiding

@Override

```
public DoctorPublicResponse getApprovedDoctorById(Long doctorId) {  
    Doctor doctor = doctorRepository.findById(doctorId)  
        .orElseThrow(() -> new ResourceNotFoundException("Doctor", "id", doctorId));  
  
    // If not approved, pretend they don't exist (privacy + clarity)  
    if (doctor.getStatus() != DoctorStatus.APPROVED) {  
        throw new ResourceNotFoundException("Doctor", "id", doctorId);  
    }  
  
    return doctorMapper.toPublicResponse(doctor);  
}
```

Why this matters » Returning 403 reveals that doctor 123 exists but is hidden. An attacker can enumerate IDs and learn which IDs map to real (but rejected/suspended) doctors. Returning 404 keeps the existence secret. **Fail closed. Leak nothing.**

INTERVIEW POINT » Interview Q: "What's the difference between 401, 403, and 404 in REST APIs?"

A: "401 Unauthorized means you're not authenticated - you need to login. 403 Forbidden means you're authenticated but lack permission. 404 Not Found means the resource doesn't exist. For privacy, we sometimes return 404 instead of 403 to hide the existence of forbidden resources. This is called resource hiding and is an OWASP recommendation."

21. Transaction Management

@Transactional Annotation

Transaction annotations

```
@Transactional                                // For writes
public DoctorResponse createProfile(...) { ... }

@Transactional(readOnly = true)                // For reads - performance hint
public DoctorResponse getMyProfile(...) { ... }
```

What @Transactional Does

- Starts a database transaction at method start
- Commits the transaction at method end (if no exception)
- Rolls back on RuntimeException (by default)
- Enables JPA dirty checking
- Manages connection lifecycle (open and close)

readOnly = true Benefits

- Hibernate skips dirty checking (faster)
- Database knows not to acquire write locks
- Some DBs route read-only TX to replicas (load balancing)
- Documents intent for code readers

COMMON MISTAKE » Default rollback: only RuntimeException triggers rollback. Checked exceptions (Exception, IOException, etc.) do NOT trigger rollback by default! Use @Transactional(rollbackFor = Exception.class) if you throw checked exceptions and want rollback.

PART V · CONTROLLER LAYER

22. Three Controllers Pattern

Why this matters » Single Responsibility Principle. Each controller serves ONE audience. Mixing them creates confusing routing, complex security, and unclear method organization. Three small focused controllers beat one giant controller.

Our Three Controllers

Controller	Base Path	Role	Concern
DoctorController	/doctors/me	DOCTOR	Self-service
AdminDoctorController	/admin/doctors	ADMIN	Approval workflow
PublicDoctorController	/doctors	PUBLIC	Browsing

Path Separation Strategy

Path strategy

```
/doctors                <- public (browse, search)
/doctors/{id}           <- public (view one)
/doctors/search         <- public (search)
/doctors/specializations <- public (filter options)
/doctors/me             <- doctor self-service
/doctors/me/resubmit    <- doctor self-service

/admin/doctors          <- admin only
/admin/doctors/{id}     <- admin only
/admin/doctors/{id}/... <- admin only
```

Clear paths = clear intent.

23. DoctorController (Self-Service)

DoctorController structure

```
@RestController
@RequestMapping("/doctors/me")
@PreAuthorize("hasRole('DOCTOR')")    // Class-level - applies to ALL methods!
public class DoctorController {

    private final DoctorService doctorService;
    private final UserService userService;

    @PostMapping
    public ResponseEntity<ApiResponse<DoctorResponse>> createProfile(
        @Valid @RequestBody DoctorProfileCreateRequest request,
        @AuthenticationPrincipal UserDetails userDetails) {
        User user = userService.findByEmail(userDetails.getUsername());
        DoctorResponse response = doctorService.createProfile(request, user);
        return new ResponseEntity<>(
            ApiResponse.success("Doctor profile created.", response),
            HttpStatus.CREATED);
    }
    // ... GET, PUT methods
}
```

Key Annotations

Annotation	Purpose
@RestController	Marks as REST controller (auto-JSON)
@RequestMapping(...)	Base path for all methods in class
@PreAuthorize at class level	Applies role check to ALL methods (DRY)
@Valid	Trigger Bean Validation
@RequestBody	Parse JSON body to Java object
@AuthenticationPrincipal	Auto-inject authenticated user
ResponseEntity<T>	Control HTTP status + body

TIP » Class-level @PreAuthorize is more DRY than putting it on every method. But if some methods need different roles, you can override at method level.

24. AdminDoctorController

Approve Endpoint Example

Approve endpoint

```
@PutMapping("/{id}/approve")
public ResponseEntity<ApiResponse<DoctorResponse>> approveDoctor(
    @PathVariable Long id,
    @Valid @RequestBody(required = false) DoctorApprovalRequest request,
    @AuthenticationPrincipal UserDetails userDetails) {

    User adminUser = userService.findByEmail(userDetails.getUsername());

    // Body is optional - create empty if not provided
    if (request == null) {
        request = new DoctorApprovalRequest();
    }

    DoctorResponse response = doctorService.approveDoctor(id, request, adminUser);
    return ResponseEntity.ok(
        ApiResponse.success("Doctor approved successfully", response));
}
```

@RequestBody(required = false)

Approval body is OPTIONAL (admin can approve without notes). Setting required=false allows the request body to be null. We then create an empty DTO in code if needed.

List by Status with Pagination

List endpoint with pagination

```
@GetMapping
public ResponseEntity<ApiResponse<Page<DoctorSummaryResponse>>> getDoctorsByStatus(
    @RequestParam(defaultValue = "PENDING_APPROVAL") DoctorStatus status,
    @PageableDefault(size = 10, sort = "createdAt",
        direction = Sort.Direction.DESC) Pageable pageable) {

    Page<DoctorSummaryResponse> doctors = doctorService.getDoctorsByStatus(status, pageable);
    return ResponseEntity.ok(ApiResponse.success("Doctors retrieved", doctors));
}
```

TIP » Spring auto-converts enum values from query string. ?status=APPROVED becomes DoctorStatus.APPROVED. If invalid, throws MethodArgumentTypeMismatchException.

25. PublicDoctorController

PublicDoctorController

```
@RestController
@RequestMapping("/doctors")
// NO @PreAuthorize - public access
public class PublicDoctorController {

    private final DoctorService doctorService;

    @GetMapping
    public ResponseEntity<ApiResponse<Page<DoctorPublicResponse>>> getApprovedDoctors(
        @PageableDefault(size = 10, sort = "fullName",
            direction = Sort.Direction.ASC) Pageable pageable) {
        Page<DoctorPublicResponse> doctors = doctorService.getApprovedDoctors(pageable);
        return ResponseEntity.ok(ApiResponse.success("Doctors retrieved", doctors));
    }

    @GetMapping("/{id}")
    public ResponseEntity<ApiResponse<DoctorPublicResponse>> getDoctorById(
        @PathVariable Long id) {
        DoctorPublicResponse response = doctorService.getApprovedDoctorById(id);
        return ResponseEntity.ok(ApiResponse.success("Doctor details retrieved", response));
    }

    @GetMapping("/search")
    public ResponseEntity<ApiResponse<Page<DoctorPublicResponse>>> searchDoctors(
        @RequestParam(required = false) String name,
        @RequestParam(required = false) String specialization,
        @PageableDefault(size = 10) Pageable pageable) {
        Page<DoctorPublicResponse> doctors =
            doctorService.searchApprovedDoctors(name, specialization, pageable);
        return ResponseEntity.ok(ApiResponse.success("Search results", doctors));
    }

    @GetMapping("/specializations")
    public ResponseEntity<ApiResponse<List<String>>> getSpecializations() {
        List<String> specializations = doctorService.getDistinctSpecializations();
        return ResponseEntity.ok(ApiResponse.success("Specializations", specializations));
    }
}
```

WARNING » Critical: Public access is granted in SecurityConfig.java, not by missing @PreAuthorize. By default, all endpoints require authentication. SecurityConfig must explicitly permit /doctors GETs.

26. Pagination with Pageable

Why this matters » Returning ALL doctors in one response is a disaster. 100K doctors = 100MB JSON = slow, expensive, and crashes browsers. Pagination is non-negotiable for list endpoints.

Query Parameter Convention

Pagination params

GET /doctors?page=0&size=10&sort=fullName,asc

Parameters:

- page 0-indexed page number (first page = 0)
- size items per page
- sort field,direction (asc or desc)
 - Multiple sorts: ?sort=specialization,asc&sort=fullName,asc

Spring auto-converts to:

```
Pageable {
    pageNumber: 0,
    pageSize: 10,
    sort: Sort.by("fullName").ascending()
}
```

Response:

```
Page<DoctorResponse> {
    content: [...10 items...],
    totalElements: 247,
    totalPages: 25,
    number: 0,
    size: 10,
    first: true,
    last: false,
    empty: false
}
```

Setting Defaults with @PageableDefault

@PageableDefault

@GetMapping

```
public ResponseEntity<...> list(
    @PageableDefault(
        size = 10,                // Default items per page
        sort = "fullName",        // Default sort field
        direction = Sort.Direction.ASC // Default direction
    ) Pageable pageable) {
    // ...
}
```

TIP » Use @PageableDefault with sensible defaults. If a client forgets ?size=10, they'll get 10 items not all 247. Limits prevent accidental DOS attacks via huge result sets.

PART VI · SECURITY UPDATES

27. SecurityConfig Changes

The Updated authorizeHttpRequests

Updated security rules

```
.authorizeHttpRequests(auth -> auth
    // PUBLIC endpoints (no auth)
    .requestMatchers("/auth/**").permitAll()
    .requestMatchers("/actuator/**").permitAll()

    // Public doctor browsing - GET only!
    .requestMatchers(HttpMethod.GET, "/doctors").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/search").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/specializations").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/{id:[0-9]+}").permitAll()

    // Everything else requires auth
    .anyRequest().authenticated()
)
```

Why HttpMethod.GET Specifically?

Method-specific rules

WITHOUT HttpMethod.GET:
 .requestMatchers("/doctors").permitAll()
 -> ALL methods (GET, POST, PUT, DELETE) on /doctors are public
 -> CATASTROPHE: anyone can create/update/delete!

WITH HttpMethod.GET:
 .requestMatchers(HttpMethod.GET, "/doctors").permitAll()
 -> Only GET is public
 -> POST, PUT, DELETE still require authentication (correctly)

COMMON MISTAKE » Forgetting HttpMethod.GET is a severe security mistake. Always specify the HTTP method when granting public access. Default to most restrictive.

28. Method-Level vs Class-Level @PreAuthorize

Class-Level (DRY)

Class-level

```
@RestController
@RequestMapping("/admin/doctors")
@PreAuthorize("hasRole('ADMIN')") // Applies to ALL methods
public class AdminDoctorController {

    @GetMapping("/{id}")
    public ... getDoctorById(...) { ... } // ADMIN required (inherited)

    @PutMapping("/{id}/approve")
    public ... approveDoctor(...) { ... } // ADMIN required (inherited)
}
```

Method-Level (Granular)

Method-level

```
@RestController
@RequestMapping("/doctors")
public class MixedDoctorController {

    @GetMapping
    public ... listDoctors() { ... } // No annotation = public

    @PostMapping
    @PreAuthorize("hasRole('DOCTOR')") // DOCTOR only
    public ... createProfile(...) { ... }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')") // ADMIN only
    public ... deleteDoctor(...) { ... }
}
```

Method-Level Overrides Class-Level

Override pattern

```
@RestController
@PreAuthorize("hasRole('DOCTOR')") // Default: DOCTOR
public class SomeController {

    @PutMapping("/special")
    @PreAuthorize("hasAnyRole('DOCTOR', 'ADMIN')") // Override: DOCTOR or ADMIN
    public ... specialAction(...) { ... }
}
```

TIP » Use class-level for consistency, method-level for exceptions. The goal: minimize repetition while keeping security explicit.

29. Regex in URL Patterns

The Problem

The conflict

Path conflicts without regex:

```
/doctors/{id}          <- matches /doctors/search too!
/doctors/search         <- separate endpoint
```

When request: GET /doctors/search
Spring tries to match /doctors/{id} first
id = "search"
Then looks up doctor with id=search -> error!

Solution: Regex Constraint

Regex constraint

```
.requestMatchers(HttpMethod.GET, "/doctors/{id:[0-9]+}").permitAll()
//                                     ^^^^^^^^^^^^^^^^^
//                                     Only numeric IDs match
```

How It Works

Regex matching

Pattern: /doctors/{id:[0-9]+}

[0-9]+ means: one or more digits

```
Matches:   /doctors/1          ID = 1
           /doctors/42         ID = 42
           /doctors/12345      ID = 12345
```

DOES NOT match:

```
/doctors/search      (not numeric)
/doctors/specializations
/doctors/abc
```

So /doctors/search routes to a different handler.
No conflict.

TIP » Regex in path variables can be used in @PathVariable too:
@GetMapping("/{id:[0-9]+}") - only match numeric IDs

PART VII · TESTING & VERIFICATION

30. Test Strategy

End-to-End Manual Testing

Phase 2 testing was manual via Postman. We simulated the complete real-world workflow with three personas:

- **Dr. Smith (Doctor):** Registers, creates profile, handles rejection, resubmits
- **Admin User:** Reviews, rejects, approves, suspends, reactivates
- **Public Patient:** Browses (no auth), searches, views details

Why Manual First?

- Catches integration bugs (everything wired correctly?)
- Validates the user journey works
- Confirms HTTP status codes and JSON shapes
- Reveals security gaps (PATIENT can access DOCTOR endpoints?)

Automated Tests (Future Phase)

Manual testing is the floor, not the ceiling. In future phases we'll add: JUnit 5 unit tests for services, Mockito for mocking, Spring Boot Test for integration tests, Testcontainers for real DB testing, MockMvc for controller tests. For Phase 2 learning, manual testing validates the workflow.

31. Complete Postman Test Suite

Test Matrix (20 Tests)

#	Test Case	Method	Endpoint	Expected
1.1	Register admin	POST	/auth/register	201
1.2	Register doctor	POST	/auth/register	201
1.3	Admin login	POST	/auth/login	200 + JWT
1.4	Doctor login	POST	/auth/login	200 + JWT
2.1	Create profile	POST	/doctors/me	201, PENDING
2.2	Public hidden	GET	/doctors	200, empty
2.3	View own	GET	/doctors/me	200
2.4	Admin sees pending	GET	/admin/doctors	200
3.1	Admin rejects	PUT	/admin/doctors/1/reject	200, REJECTED
3.2	Doctor sees reject	GET	/doctors/me	200
3.3	Update blocked	PUT	/doctors/me	400
3.4	Resubmit	PUT	/doctors/me/resubmit	200, PENDING
4.1	Admin approves	PUT	/admin/doctors/1/approve	200, APPROVED
4.2	Public sees	GET	/doctors	200, 1 doctor
4.3	Search by name	GET	/doctors/search?name=	200
4.4	Search by spec	GET	/doctors/search?spec=	200
4.5	Specializations	GET	/doctors/specializations	200
5.1	Suspend	PUT	/admin/doctors/1/suspend	200, SUSPENDED
5.2	Public hidden	GET	/doctors	200, empty
5.3	Reactivate	PUT	/admin/doctors/1/reactivate	200, APPROVED

Security Tests

Test	Description	Expected
B1	PATIENT calls POST /doctors/me	403 Forbidden
B2	DOCTOR calls GET /admin/doctors	403 Forbidden
B3	No auth on POST /doctors/me	403 Forbidden

Test	Description	Expected
B4	Expired JWT	403 Forbidden
B5	Invalid JWT signature	403 Forbidden

32. MySQL Verification

Verify Schema Created

Schema check

```
USE medibook_db;
SHOW TABLES;
-- Expected: 'doctors' and 'users'

DESCRIBE doctors;
-- Should show all 19 columns
```

Verify Indexes

Index check

```
SHOW INDEX FROM doctors;

-- Should show indexes:
-- PRIMARY (id)
-- idx_doctor_user (user_id, UNIQUE)
-- idx_doctor_license (license_number, UNIQUE)
-- idx_doctor_specialization (specialization)
-- idx_doctor_status (status)
```

Verify Foreign Key

FK check

```
SELECT
    TABLE_NAME, COLUMN_NAME, CONSTRAINT_NAME,
    REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'medibook_db'
    AND TABLE_NAME = 'doctors'
    AND REFERENCED_TABLE_NAME IS NOT NULL;

-- Should show: doctors.user_id -> users.id
```

Watch State Transitions Live

Live monitoring

```
-- Run during testing to see state changes
SELECT
    id, full_name, status,
    rejection_reason, resubmission_allowed,
    approved_by_user_id, approved_at,
    rejected_by_user_id, rejected_at
FROM doctors;

-- Run after each Postman call to see DB updates
```

PART VIII · REFERENCE

33. All Endpoints Cheat Sheet

Method	Endpoint	Auth	Role	Description
POST	/auth/register	No	Public	Register user
POST	/auth/login	No	Public	Login, get JWT
GET	/users/me	Yes	Any	Current user
POST	/doctors/me	Yes	DOCTOR	Create profile
GET	/doctors/me	Yes	DOCTOR	View own profile
PUT	/doctors/me	Yes	DOCTOR	Update profile
PUT	/doctors/me/resubmit	Yes	DOCTOR	Resubmit after rejection
GET	/admin/doctors	Yes	ADMIN	List by status
GET	/admin/doctors/{id}	Yes	ADMIN	View any doctor
PUT	/admin/doctors/{id}/approve	Yes	ADMIN	Approve doctor
PUT	/admin/doctors/{id}/reject	Yes	ADMIN	Reject with reason
PUT	/admin/doctors/{id}/suspend	Yes	ADMIN	Suspend
PUT	/admin/doctors/{id}/reactivate	Yes	ADMIN	Reactivate
GET	/doctors	No	Public	Browse approved
GET	/doctors/{id}	No	Public	Doctor details
GET	/doctors/search	No	Public	Search
GET	/doctors/specializations	No	Public	Filter options

34. Common Mistakes & Fixes

Mistake	Symptom	Fix
Missing @Service on impl	NoSuchBeanDefinitionException	Add @Service
Missing @Transactional	Updates don't persist	Add @Transactional
Using EnumType.ORDINAL	Data corruption on reorder	Always EnumType.STRING
double for money	Rounding errors	Use BigDecimal
Forgetting @JoinColumn unique	Multiple doctors per user	Add unique=true
Public access without HttpMethod	All HTTP methods public	Specify HttpMethod
@PreAuthorize without @EnableMethodSecurity	Not secured	Enable in config
Returning entity from service	Lazy loading breaks	Return DTO
Not validating state transitions	Invalid state changes	Check status before update
Returning 403 for hidden resources	Reveals existence	Return 404
Forgetting Pageable parameter	Returns ALL records	Always paginate lists
Path conflicts (/doctors/{id} vs /doctors/{id}/search)	Wrong path matched	Use regex constraint

35. Interview Questions for Phase 2

Q1: Walk me through your doctor approval workflow

INTERVIEW POINT » "MediBook implements a state machine for doctor onboarding. Doctors self-register as users with the DOCTOR role, then create a professional profile through POST /doctors/me. The profile starts in PENDING_APPROVAL state. Admins review pending doctors via GET /admin/doctors?status=PENDING_APPROVAL, then either approve (sets APPROVED status) or reject (sets REJECTED with reason). Rejected doctors can resubmit if admin allows it. Only APPROVED doctors are visible in the public /doctors listing. All state transitions are validated in the service layer to prevent invalid changes. The system tracks audit information: who approved/rejected and when, for compliance."

Q2: How did you handle role-based access?

INTERVIEW POINT » "Three layers. First, Spring Security with JWT validates the user. Second, SecurityConfig defines which URL patterns require auth and which are public - I used HttpMethod.GET to allow public reads while keeping writes protected. Third, @PreAuthorize annotations on controllers enforce role-based access. I used class-level @PreAuthorize for consistency since each controller has one target audience. DoctorController requires DOCTOR role, AdminDoctorController requires ADMIN role, PublicDoctorController is open. This separation makes the API contract obvious."

Q3: How did you implement pagination?

INTERVIEW POINT » "Spring Data's Pageable interface. I declare Pageable as a controller parameter with @PageableDefault to set defaults like size=10 and sort=fullName,asc. Spring auto-binds query params like ?page=0&size=20 to the Pageable. The repository accepts Pageable and returns Page, which includes content plus metadata (totalElements, totalPages, etc.). I never write LIMIT/OFFSET SQL - Spring handles it. This works for sorting, filtering, and partial result sets, all dynamically."

Q4: How did you handle state transitions?

INTERVIEW POINT » "In the service layer, I validate current state BEFORE applying changes. For example, approveDoctor() first checks if status == PENDING_APPROVAL. If not, throws BadRequestException with a clear message. This pattern is in every transition method - approve, reject, suspend, reactivate, resubmit. Centralizing in the service prevents invalid changes regardless of which API endpoint calls it. For larger systems, I'd extract this into a dedicated StateMachine class, possibly using Spring State Machine library."

Q5: Why 7 different DTOs for one entity?

INTERVIEW POINT » "Different consumers need different views. CreateRequest has licenseNumber (set once); UpdateRequest doesn't (immutable). Patients see DoctorPublicResponse (no phone/license/status); admins see DoctorResponse (everything). List views use DoctorSummaryResponse (minimal data for performance). Forcing all consumers to use one big DTO would expose data they shouldn't see (security risk) and waste bandwidth on unused fields. This is Interface Segregation Principle applied at the API layer."

Q6: How did you handle the case where doctor exists but isn't visible?

INTERVIEW POINT » "Resource hiding. For a non-APPROVED doctor accessed via the public endpoint, I throw ResourceNotFoundException which returns 404. If I returned 403 instead, attackers could enumerate IDs and learn which exist but are hidden. 404 hides existence entirely. This is OWASP's fail-closed principle - leak nothing. For admin endpoints, I return the doctor regardless of status because admins need to see everything for management purposes."

36. What's Next: Phase 3 Preview

Phase 3: Slot Management

Building on Phase 2, the next phase implements time slots for appointments.

New Concepts You'll Learn

- **Time-based logic:** LocalDate, LocalTime, ZoneId
- **Recurring patterns:** Doctor works Mon-Fri 10am-8pm
- **Bulk operations:** Generate slots for 30 days at once
- **Configuration:** Per-doctor slot rules
- **Holiday handling:** Skip slot generation for holidays
- **Scheduled tasks:** @Scheduled to auto-generate future slots

Endpoints Coming

Method	Endpoint	Purpose
POST	/admin/doctors/{id}/slot-config	Configure slot rules
POST	/doctors/me/slots/generate	Generate slots
GET	/doctors/{id}/slots	View available slots
DELETE	/doctors/me/slots/{id}	Cancel a slot

Concurrency Challenge in Phase 4

Phase 4 (Booking) will introduce concurrency control. When 2 patients click "book" on the same slot at the same millisecond, only ONE should succeed. We'll use pessimistic locking, optimistic locking, or database constraints. This is a classic distributed systems problem - great interview material.

PHASE 2 COMPLETE!

You now have a production-grade doctor module with:

Approval workflow · State machine · Audit trail

Pagination · Search · Role-based access

Repository: github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook

Phase 3 awaits!